

INTRODUÇÃO E SINTAXE BÁSICA JAVA

Prof. Me. Hélio Esperidião

```
mirror_mod = modifier_ob.  
set mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly  
-- OPERATOR CLASSES --  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
context):  
context.active_object is not
```

Worldwide, Jul 2025 :

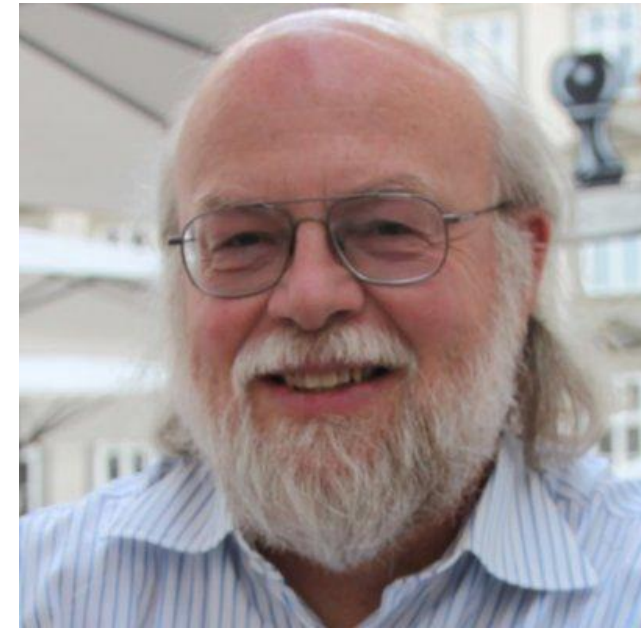
Rank	Change	Language	Share	1-year trend
1		Python	31.47 %	+1.9 %
2		Java	15.22 %	-0.2 %
3		JavaScript	7.65 %	-0.7 %
4	↑	C/C++	7.05 %	+0.8 %
5	↓	C#	5.81 %	-0.9 %
6		R	4.88 %	+0.3 %
7		PHP	3.55 %	-0.8 %
8		TypeScript	2.76 %	-0.2 %
9	↑↑	Objective-C	2.76 %	+0.4 %
10		Rust	2.72 %	+0.1 %
11	↓↓	Swift	2.19 %	-0.5 %
12		Go	1.92 %	-0.2 %
13	↑↑↑↑↑↑↑	Ada	1.65 %	+0.7 %
14	↓	Kotlin	1.59 %	-0.3 %
15	↓	Matlab	1.57 %	+0.1 %
16		Powershell	1.03 %	+0.0 %
17	↑	Ruby	0.95 %	+0.0 %
18	↓↓↓	VBA	0.95 %	-0.1 %
19	↓↓	Dart	0.89 %	-0.1 %
20		Lua	0.89 %	+0.1 %

JAVA NO MUNDO

- Github
- <https://pypl.github.io/PYPL.html>

1990

- No início dos anos **1990**, um grupo da Sun, liderado por **James Gosling**, desenvolveu uma versão mais simples do C++, que foi batizada de **Java**.
- O grupo esperava que o Java fosse usado para aplicações de vídeo sob demanda, mas o projeto não evoluiu e tomou outro rumo.



Java Software: Metas

Simples

Orientada a Objetos

Familiar

Portável e Independente de máquina

- Programas em java rodam em
 - Servidores
 - Desktops
 - Celulares
 - Cartões
- Usa bytecodes

Java Virtual Machine (JVM) Máquina virtual Java

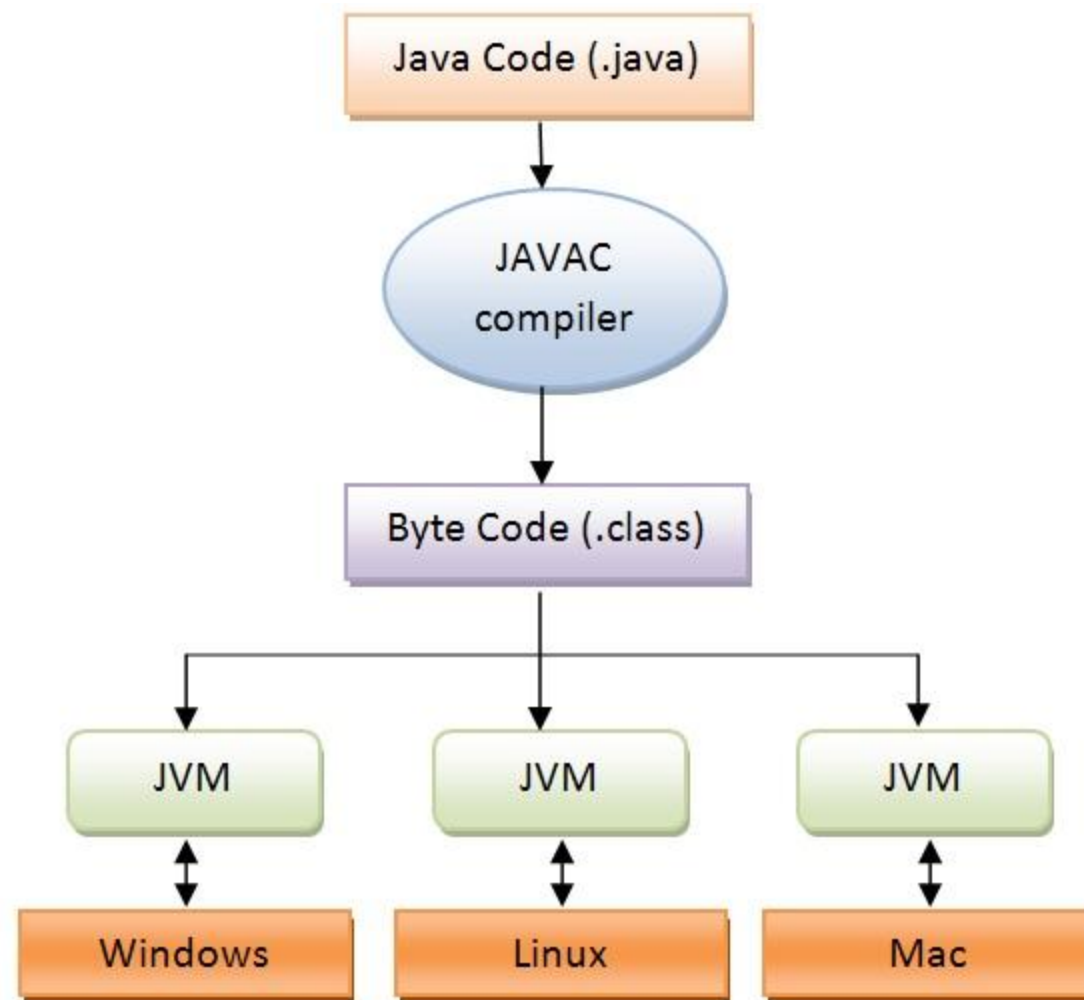
É uma máquina virtual que carrega e executa os aplicativos Java.

Convertendo os *bytecodes* em código executável de máquina.

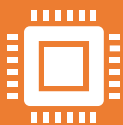
A JVM é responsável pelo gerenciamento dos aplicativos, à medida que são executados.

Graças à máquina virtual Java, os programas escritos em Java podem funcionar em qualquer plataforma de hardware e software que possua uma versão da JVM,

Tornando as aplicações independentes da plataforma onde funcionam.



JDK - *Java Development Kit*



É um pacote de software que você pode usar para desenvolver aplicativos baseados em Java.



O Java Development Kit é necessário para desenvolver aplicativos java.



O Java JDK é composto pelo compilador e pelas bibliotecas (API's) necessárias para criação de programas em Java e ferramentas úteis para o desenvolvimento e para testes dos programas escritos por esta linguagem de programação

JRE - *Java Runtime Environment*



O Java Runtime Environment (JRE) é um software necessário para que os programas em Java funcionem corretamente.



O JRE é a tecnologia subjacente que estabelece a comunicação entre o programa em Java e o sistema operacional.



Atua como um tradutor e facilitador, fornecendo todos os recursos para que o software que você cria usando a linguagem funcione em qualquer sistema operacional sem modificações adicionais.

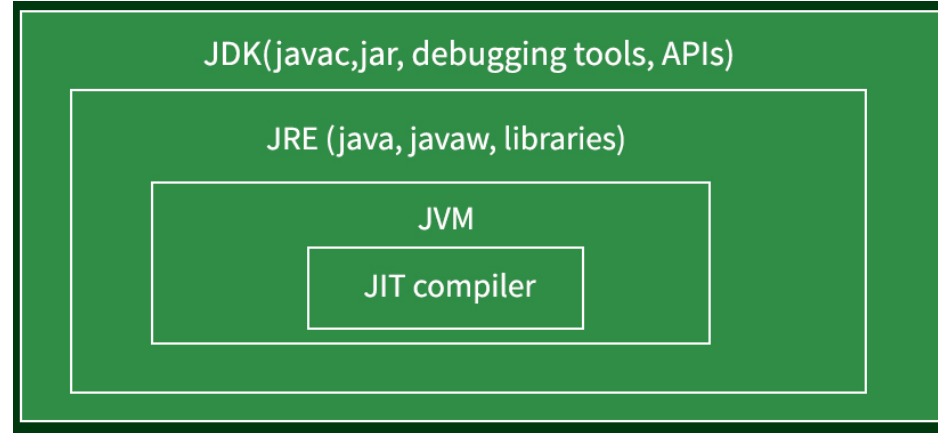
A diferença entre o JRE, a JVM e o JDK

No JDK, você escreve o código usando sintaxe semelhante ao idioma inglês.

O JDK compila o código e o transmite em bytes para o JRE.

Por sua vez, o JRE contém bibliotecas de classes, arquivos de suporte e a JVM.

- Ele usa esses componentes de software para executar o código em bytes em qualquer dispositivo.



JIT

- O compilador Just-In-Time (JIT) é um componente do Java™ Runtime Environment.
- Ele melhora o desempenho dos aplicativos Java compilando bytecodes para código de máquina nativo no tempo de execução.



VM vs JIT

VM	JIT
JVM significa Java Virtual Machine.	JIT significa compilação Just-in-time.
A JVM foi introduzida para gerenciar a memória do sistema e fornecer um ambiente de execução transportável para aplicativos baseados em Java	O JIT foi inventado para melhorar o desempenho da JVM após muitos anos de seu lançamento inicial.
JVM consiste em muitos outros componentes, como área de pilha, área de heap, etc.	JIT é um dos componentes da JVM.
JVM compila código de byte completo para código de máquina.	O JIT compila apenas o código de byte reutilizável em código de máquina.
JVM fornece independência de plataforma.	JIT melhora o desempenho da JVM.

Primeiro programa java

```
// Declaração da classe principal do programa
class Exemplo01 {

    // Método principal: ponto de entrada do programa
    public static void main(String[] args) {
        // Exibe a mensagem "Ola Mundo :)" no console
        System.out.println("Ola Mundo :)");
    }
}
```

Primitivos

- Em Java, **primitivos** são os tipos de dados mais básicos da linguagem. Eles não são objetos, ocupam menos memória e têm desempenho melhor, porque armazenam diretamente o valor, e não uma referência.

Primitivos em java

Tipo	Tamanho	Valor padrão	Faixa de valores	Exemplo
byte	8 bits	0	-128 a 127	byte b = 10;
short	16 bits	0	-32.768 a 32.767	short s = 200;
int	32 bits	0	-2.147.483.648 a 2.147.483.647	int i = 1000;
long	64 bits	0L	-9.223.372.036.854.775.808 a 9.223...	long l = 100L;
float	32 bits	0.0f	±3.4E38 (aprox.)	float f = 3.14f;
double	64 bits	0.0d	±1.7E308 (aprox.)	double d = 3.14;
char	16 bits	'\u0000'	0 a 65.535 (Unicode)	char c = 'A';
boolean	1 bit	false	true ou false	boolean b = true;

Declaração de variáveis

Dentro de um bloco, podemos declarar variáveis e usá-las.

Em Java, toda variável tem um tipo que não pode ser mudado uma vez declarado:

***tipoDaVariavel* nomeDaVariavel;**

```
public class Exemplo02 {  
    public static void main(String[] args) {  
        int x = 10;  
        byte numeroByte = 100; // byte: 8 bits, de -128 a 127  
        short numeroShort = 20000; // short: 16 bits, de -32.768 a 32.767  
        int numeroInt = 100000; // int: 32 bits, de -2.147.483.648 a 2.147.483.647  
        long numeroLong = 100000000000L; // long: 64 bits, precisa do sufixo 'L'  
        // Tipos de ponto flutuante  
        float numeroFloat = 3.14f; // float: 32 bits, precisa do sufixo 'f'  
        double numeroDouble = 3.1415926535; // double: 64 bits, maior precisão  
        // Tipo caractere  
        char letra = 'A'; // char: 16 bits, armazena um único caractere Unicode  
        // Tipo lógico  
        boolean ativo = true; // boolean: representa verdadeiro ou falso  
        // Impressão dos valores  
        System.out.println("byte: " + numeroByte);  
        System.out.println("short: " + numeroShort);  
        System.out.println("int: " + numeroInt);  
        System.out.println("long: " + numeroLong);  
        System.out.println("float: " + numeroFloat);  
        System.out.println("double: " + numeroDouble);  
        System.out.println("char: " + letra);  
        System.out.println("boolean: " + ativo);  
    }  
}
```


Exemplo String

```
public class Exemplo03 {  
    public static void main(String[] args) {  
        // Declaração simples  
        String nome = "Hélio"; // Criação direta (mais usada)  
  
        // Criação usando o operador new  
        String saudacao = new String("Olá"); // Menos usada  
  
        // Concatenação  
        String mensagem = saudacao + " " + nome + "!"; // Une várias strings  
  
        // Exibição no console  
        System.out.println(mensagem); // Saída: Olá Hélio!  
    }  
}
```

Conversão entre tipos

Tipo	conversão
double	<code>Double.parseDouble(String);</code>
float	<code>Float.parseFloat(String);</code>
long	<code>Long.parseLong(String);</code>
int	<code>Integer.parseInt(String);</code>
char	<code>stringGoesHere.charAt(int position);</code>
short	<code>Short.parseShort(String);</code>
byte	<code>Byte.parseByte(String);</code>
boolean	<code>Boolean.parseBoolean(String);</code>

```
class Exemplo04 {  
    public static void main(String[] args) {  
        // Exemplo de string contendo número  
        String texto = "15";  
  
        // Conversões de String para tipos primitivos  
        byte valorByte = Byte.parseByte(texto); // String → byte  
        short valorShort = Short.parseShort(texto); // String → short  
        int valorInt = Integer.parseInt(texto); // String → int  
        long valorLong = Long.parseLong(texto); // String → long  
        float valorFloat = Float.parseFloat(texto); // String → float  
        double valorDouble = Double.parseDouble(texto); // String → double  
        boolean valorBoolean = Boolean.parseBoolean("true"); // String → boolean  
        String valorChar = Character.toString('A'); // char → string  
  
        // Impressão dos valores convertidos  
        System.out.println("String para byte: " + valorByte);  
        System.out.println("String para short: " + valorShort);  
        System.out.println("String para int: " + valorInt);  
        System.out.println("String para long: " + valorLong);  
        System.out.println("String para float: " + valorFloat);  
        System.out.println("String para double: " + valorDouble);  
        System.out.println("String para boolean: " + valorBoolean);  
        System.out.println("String para char: " + valorChar);  
    }  
}
```

```

public class Exemplo05 {
    public static void main(String[] args) {
        String texto = "16";

        byte valorByte = Byte.parseByte(texto); // String → byte
        short valorShort = Short.parseShort(texto); // String → short
        int valorInt = Integer.parseInt(texto); // String → int
        long valorLong = Long.parseLong(texto); // String → long
        float valorFloat = Float.parseFloat(texto); // String → float
        double valorDouble = Double.parseDouble(texto); // String → double
        boolean valorBoolean = Boolean.parseBoolean("true"); // String → boolean
        String valorChar = Character.toString('A'); // char → string

        // Conversões de primitivos para String
        String deByte = Byte.toString(valorByte);
        String deShort = Short.toString(valorShort);
        String deInt = Integer.toString(valorInt);
        String deLong = Long.toString(valorLong);
        String deFloat = Float.toString(valorFloat);
        String deDouble = Double.toString(valorDouble);
        String deBoolean = Boolean.toString(valorBoolean);
        String deChar = valorChar;

        // Impressão dos valores convertidos de volta para String
        System.out.println("\nPrimitivos para String:");
        System.out.println("byte para String: " + deByte);
        System.out.println("short para String: " + deShort);
        System.out.println("int para String: " + deInt);
        System.out.println("long para String: " + deLong);
        System.out.println("float para String: " + deFloat);
        System.out.println("double para String: " + deDouble);
        System.out.println("boolean para String: " + deBoolean);
        System.out.println("char para String: " + deChar);
    }
}

```

Entrada de dados



A classe Scanner tem como objetivo separar a entrada dos textos em blocos, gerando os conhecidos tokens, que são sequências de caracteres separados por delimitadores que por padrão correspondem aos espaços em branco, tabulações e mudança de linha.



Com essa classe podem ser convertidos textos para tipos primitivos, sendo que esses textos podem ser considerados como objetos do tipo String, InputStream e arquivos.

Principais métodos da classe Scanner



Método	Função
nextFloat()	Ler float
nextInt()	Lert int
nextByte()	Ler byte
nextLong()	Ler long
nextBoolean()	Ler booleano
nextDouble()	Ler double
nextLine()	Ler linha inteira - String

nextFloat()

```
// Importa a classe Scanner, usada para ler dados digitados pelo usuário
import java.util.Scanner;

// Classe principal do programa
class Exemplo06 {
    // Método principal: ponto de entrada do programa
    public static void main(String[] args) {
        // Cria um objeto da classe Scanner que vai ler dados da entrada padrão
        Scanner entradaDados = new Scanner(System.in);

        System.out.println("Digite um número do tipo long:");
        // Lê um número do tipo long digitado pelo usuário e armazena na variável
        long dado = entradaDados.nextLong();

        // Exibe no console a mensagem "Dado lido: " seguida do valor armazenado
        System.out.println("Dado lido: " + dado);

        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

nextInt()

```
// Importa a classe Scanner do pacote java.util, que permite ler entrada de dados
import java.util.Scanner;

// Declaração da classe chamada Exemplo07
class Exemplo07 {
    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        System.out.println("Digite um número inteiro:");
        Scanner entradaDados = new Scanner(System.in);
        // Lê um valor inteiro da entrada padrão e armazena na variável 'dado'
        int dado = entradaDados.nextInt();
        // Imprime o valor lido concatenado com a string "Dado lido: "
        System.out.println("Dado lido: " + dado);
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```


Ler byte

```
// Importa a classe Scanner, usada para ler dados do teclado
import java.util.Scanner;
class Exemplo08 {
    public static void main(String[] args) {
        // Cria um objeto Scanner para capturar entradas do usuário pelo console
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um byte: ");
        // Lê um valor do tipo byte digitado pelo usuário e armazena na variável 'dado'
        byte dado = entradaDados.nextByte();
        // Exibe no console a mensagem com o valor lido
        System.out.println("Dado lido: " + dado);
        // Fecha o objeto Scanner para liberar recursos
        entradaDados.close();
    }
}
```

Ler Long

```
// Importa a classe Scanner, que permite a leitura de dados digitados pelo usuário
import java.util.Scanner;

class Exemplo09 {
    public static void main(String[] args) {
        // Cria um objeto Scanner para capturar dados do teclado
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um long:");
        // Lê um número do tipo long digitado pelo usuário e armazena na variável 'dado'
        long dado = entradaDados.nextLong();
        // Exibe no console a mensagem com o valor lido
        System.out.println("Dado lido: " + dado);
        // Fecha o objeto Scanner para liberar os recursos utilizados
        entradaDados.close();
    }
}
```

Ler boolean

```
// Importa a classe Scanner do pacote java.util, que permite ler entrada de dados
import java.util.Scanner;
public class Exemplo10 {
    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        // (System.in)
        Scanner entradaDados = new Scanner(System.in);
        // Exibe mensagem solicitando que o usuário digite um valor booleano
        System.out.println("Digite um boolean:");
        // Lê um valor do tipo boolean da entrada padrão e armazena na variável 'dado'
        boolean dado = entradaDados.nextBoolean();
        // Imprime o valor lido concatenado com a string "Dado lido: "
        System.out.println("Dado lido: " + dado);
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

Ler double

```
// Importa a classe Scanner do pacote java.util, que permite ler entrada de dados
import java.util.Scanner;
public class Exemplo11 {
    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        // (System.in)
        Scanner entradaDados = new Scanner(System.in);
        // Exibe mensagem solicitando que o usuário digite um valor do tipo double
        System.out.println("Digite um double:");
        // Lê um valor do tipo double da entrada padrão e armazena na variável 'dado'
        double dado = entradaDados.nextDouble();
        // Imprime o valor lido concatenado com a string "Dado lido: "
        System.out.println("Dado lido: " + dado);
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

Ler String

```
// Importa a classe Scanner do pacote java.util, que permite ler entrada de dados
import java.util.Scanner;
public class Exemplo12 {
    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        // (System.in)
        Scanner entradaDados = new Scanner(System.in);
        // Exibe mensagem solicitando que o usuário digite uma String
        System.out.println("Digite uma String:");
        // Lê uma linha de texto da entrada padrão e armazena na variável 'dado'
        String dado = entradaDados.nextLine();
        // Imprime o valor lido concatenado com a string "Dado lido: "
        System.out.println("Dado lido: " + dado);

        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

Expressões Aritméticas

Operador	Descrição	Hierarquia da Operação
+	Soma	3º
-	Subtração	3º
*	Multiplicação	2º
/	Divisão	2º
(%)	Resto da divisão	2º
()	Parênteses	1º 1º

Operadores de incremento e decremento

- Em Java, operadores de incremento e decremento são usados para aumentar ou diminuir o valor de uma variável numérica (geralmente int, byte, short, long, float ou double).

Operador	Nome	Descrição
++	Incremento	Aumenta o valor da variável em 1.
--	Decremento	Diminui o valor da variável em 1.

Pré-incremento e Pré-decremento

- Forma: **++variável** ou **--variável**
- Primeiro altera o valor da variável, depois a utiliza na expressão.

Pós-incremento e Pós-decremento

- Forma: `variável++` ou `variável--`
- Primeiro usa o valor da variável na expressão e depois faz a alteração.

```
// Declaração da classe chamada Exemplo13
public class Exemplo13 {
    public static void main(String[] args) {
        // Declara e inicializa a variável 'numero' com o valor 5
        int numero = 5;
        // Pré-incremento: incrementa 'numero' antes de imprimir (6)
        System.out.println(++numero); // Saída: 6
        // Imprime o valor atual de 'numero' (6)
        System.out.println(numero);
        System.out.println("-----");
        // Reinicializa a variável 'numero' com 5
        numero = 5;
        // Pós-incremento: imprime o valor atual (5), depois incrementa para 6
        System.out.println(numero++); // Saída: 5
        // Imprime o valor atualizado de 'numero' (6)
        System.out.println(numero);
        // Pós-decremento: imprime o valor atual (6), depois decrementa para 5
        System.out.println(numero--); // Saída: 6
        // Imprime o valor atualizado de 'numero' (5)
        System.out.println(numero);
    }
}
```

Operadores relacionais

Operador	Função	Exemplo	Resultado
>	Maior	5>3	?
<	Menor	5<3	?
>=	Maior ou igual	5>=6	?
<=	Menor ou igual	7<=3	?
!=	Diferente de	5!=5	?
==	Igual a	5==5	?
!	Inversor	!5==5	?

Operadores lógicos

Operador	Função
&&	E
	Ou
!	Não
^	XOR - somente se uma das condições for verdadeira, mas não ambas.

```
// Importa a classe Scanner do pacote java.util, que permite ler entrada de dados
import java.util.Scanner;

// Declaração da classe chamada Exemplo14
public class Exemplo14 {

    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {

        // Cria um objeto Scanner chamado 'entradaDados'
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite uma nota:");
        // Lê um valor do tipo double da entrada padrão e armazena na variável 'dado'
        double dado = entradaDados.nextDouble();
        if (dado > 10) {
            System.out.println("nota inválida");
        }
        else if (dado < 0) {
            System.out.println("nota inválida");
        }
        else {
            System.out.println("nota válida");
        }
        entradaDados.close();
    }
}
```

```
// Importa a classe Scanner para leitura de dados
import java.util.Scanner;

public class Exemplo15 {
    public static void main(String[] args) {
        Scanner entradaDados = new Scanner(System.in);

        System.out.println("Digite true ou false para condição A:");
        boolean condA = entradaDados.nextBoolean();

        System.out.println("Digite true ou false para condição B:");
        boolean condB = entradaDados.nextBoolean();

        // XOR: true se somente uma das condições for verdadeira
        boolean resultado = condA ^ condB;

        System.out.println("Resultado do XOR: " + resultado);

        entradaDados.close();
    }
}
```

```
import java.util.Scanner;
class Exemplo16 {
    public static void main(String[] args) {
        Scanner entradaDados = new Scanner(System.in);
        int x = entradaDados.nextInt();
        // Estrutura switch que seleciona uma ação dependendo do valor de 'x'
        switch (x) {
            // Caso o valor seja 1, imprime "Domingo"
            case 1:
                System.out.println("Domingo");
                // Interrompe o switch para não executar os próximos casos
                break;
            // Caso o valor seja 2, imprime "Segunda"
            case 2:
                System.out.println("Segunda");
                // Interrompe o switch para não executar os próximos casos
                break;
            // Caso nenhum dos casos anteriores seja atendido, executa este bloco
            default:
                System.out.println("Dia da semana inválido");
        }
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

Estruturas de repetição


```
import java.util.Scanner;
public class Exemplo17 {
    public static void main(String[] args) {
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um número inteiro: ");
        int n = entradaDados.nextInt();
        // Estrutura de repetição for que inicia com i=0, executa enquanto i <= n,
        // e incrementa i em 1 a cada iteração
        for (int i = 0; i <= n; i = i + 1) {
            // Imprime o valor atual de i no console
            System.out.println(i);
        }

        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

```
import java.util.Scanner;
public class Exemplo18 {
    public static void main(String[] args) {
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um número inteiro positivo:");
        int n = entradaDados.nextInt();
        for (int i = 0; i <= n; i++) {
            // Se i for múltiplo de 3, pula a iteração atual (não imprime)
            if (i % 3 == 0 && i != 0) {
                continue; // pula o restante do loop e vai para o próximo i
            }
            // Se i for maior ou igual a 10, encerra o loop
            if (i >= 10) {
                break; // sai do loop completamente
            }
            // Imprime o valor atual de i (que não é múltiplo de 3 e menor que 10)
            System.out.println(i);
        }
        // Fecha o Scanner para liberar recursos
        entradaDados.close();
    }
}
```

Estruturas de repetição: while

```
import java.util.Scanner;
public class Exemplo19 {
    public static void main(String[] args) {
        // Cria um Scanner para ler entrada do usuário
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um número inteiro positivo:");

        int n = entradaDados.nextInt();
        int i = 0;

        // Loop enquanto i for menor ou igual a n
        while (i <= n) {
            // Imprime o valor atual de i
            System.out.println(i);

            // Incrementa i para a próxima iteração
            i++;
        }
        entradaDados.close();
    }
}
```

Estruturas de repetição: Do-while

```
import java.util.Scanner;

public class Exemplo20 {
    public static void main(String[] args) {
        Scanner entradaDados = new Scanner(System.in);
        System.out.println("Digite um número inteiro positivo:");

        int n = entradaDados.nextInt();
        int i = 0;

        // Loop do-while executa o bloco pelo menos uma vez
        do {
            // Imprime o valor atual de i
            System.out.println(i);

            // Incrementa i para a próxima iteração
            i++;
        } while (i <= n); // Continua enquanto i for menor ou igual a n

        // Fecha o Scanner para liberar recursos
        entradaDados.close();
    }
}
```

Operadores Compostos em Java (x += n e similares)

- São operadores que realizam uma operação aritmética entre uma variável e um valor, e em seguida atualizam a variável com o resultado dessa operação.
- Eles são uma forma simplificada de escrever expressões comuns.

Exemplos

Operador composto	Equivalente a	Descrição
$x += n$	$x = x + n$	Soma n ao valor atual de x
$x -= n$	$x = x - n$	Subtrai n do valor atual de x
$x *= n$	$x = x * n$	Multiplica x por n
$x /= n$	$x = x / n$	Divide x por n
$x \% = n$	$x = x \% n$	Calcula o resto da divisão de x por n

```
public class Exemplo21 {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println("Valor inicial de x: " + x);  
  
        x += 5; // Agora x vale 15 (10 + 5)  
        System.out.println("x = " + x);  
  
        x -= 3; // Agora x vale 12 (15 - 3)  
        System.out.println("x = " + x);  
  
        x *= 2; // Agora x vale 24 (12 * 2)  
        System.out.println("x = " + x);  
  
        x /= 4; // Agora x vale 6 (24 / 4)  
        System.out.println("x = " + x);  
  
        x %= 4; // Agora x vale 2 (6 % 4)  
        System.out.println("x = " + x);  
    }  
}
```



Vetores em java

- Declaração:
 - `tipo nomeVetor[] = new tipo[tamanho];`
 - Declaração
 - `tipo nomeVetor[] = {valorPosicao0, valorPosicao1, valorPosicao2};`
-


```
import java.util.Scanner;
// Declaração da classe chamada Exemplo22
public class Exemplo22 {
    // Método principal, ponto de entrada do programa
    public static void main(String[] args) {
        // Declaração e criação de um vetor (array) de inteiros com 10 posições
        int vetor[] = new int[10];
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        // (System.in)
        Scanner entradaDados = new Scanner(System.in);
        // Loop para ler 10 números inteiros digitados pelo usuário e armazenar no vetor
        for (int i = 0; i < 10; i++) {
            vetor[i] = entradaDados.nextInt();
        }
        // Loop para imprimir os índices de 9 até 0 em ordem decrescente
        for (int i = 9; i >= 0; i--) {
            System.out.println(i);
        }
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

```
import java.util.Scanner;
public class Exemplo23 {
    public static void main(String[] args) {
        // Declaração e inicialização de um array de Strings com abreviações dos dias da
        // semana
        String semana[] = { "dom", "seg", "ter", "qua", "qui", "sex", "sab" };
        // Cria um objeto Scanner chamado 'entradaDados' que monitora a entrada padrão
        // (System.in)
        Scanner entradaDados = new Scanner(System.in);
        // Lê um número inteiro digitado pelo usuário e armazena na variável 'i'
        int i = entradaDados.nextInt();
        // Imprime o dia da semana correspondente ao índice (ajustado para começar em 0)
        System.out.println(semana[i - 1]);
        // Fecha o objeto Scanner para liberar recursos do sistema
        entradaDados.close();
    }
}
```

Estrutura for aprimorada

Sintaxe:

```
for (tipo parâmetro: nomeDoVetor) {  
    instrução;  
}
```

Esse é o for-each em Java, também chamado de enhanced for loop.

É uma forma simplificada de percorrer todos os elementos de um array ou de uma coleção (como ArrayList, HashSet etc.) sem precisar controlar índices manualmente.

```
public class Exemplo24 {  
    public static void main(String[] args) {  
        // Declara um array de inteiros  
        int numeros[] = { 10, 20, 30, 40, 50 };  
  
        // Percorre o array usando o for-each  
        for (int num : numeros) {  
            // Imprime cada elemento do array  
            System.out.println(num);  
        }  
    }  
}
```

O que isso faz?

```
public class Exemplo25 {  
    public static void main(String[] args) {  
        int v[] = new int[3];  
        v[0] = 1;  
        v[1] = -1;  
        v[2] = 7;  
        for (int i = 0; i < v.length - 1; i++) {  
            for (int j = 0; j < v.length - 1 - i; j++) {  
                if (v[j] > v[j + 1]) {  
                    int aux = v[j];  
                    v[j] = v[j + 1];  
                    v[j + 1] = aux;  
                }  
            }  
        }  
        for (int elemento : v) {  
            System.out.println(elemento);  
        }  
    }  
}
```

```

public class Exemplo25 {
    public static void main(String[] args) {
        int v[] = new int[3];
        v[0] = 1;
        v[1] = -1;
        v[2] = 7;
        // Algoritmo de ordenação Bubble Sort para ordenar o vetor em ordem crescente
        // Loop externo controla quantas vezes o processo será repetido
        for (int i = 0; i < v.length - 1; i++) {
            // Loop interno faz comparações e trocas dos elementos adjacentes
            for (int j = 0; j < v.length - 1 - i; j++) {
                // Se o elemento atual for maior que o próximo, troca-os de posição
                if (v[j] > v[j + 1]) {
                    int aux = v[j];           // Armazena temporariamente o valor de v[j]
                    v[j] = v[j + 1];         // Passa o valor menor para a posição atual
                    v[j + 1] = aux;          // Coloca o valor maior na posição seguinte
                }
            }
        }

        // Percorre o vetor usando for-each para imprimir os elementos ordenados
        for (int elemento : v) {
            System.out.println(elemento);
        }
    }
}

```

Vetores bidimensionais (ou matrizes)



A linguagem Java não fornece vetores multidimensionais, mas como um vetor pode ser declarado e ter qualquer tipo de base, é possível criar vetores de vetores (de vetores etc.), alcançando assim o mesmo efeito.



A declaração de um vetor bidimensional para inteiros, de nome "m" em Java:



Sintaxe:

```
tipo nomeMatriz[][] = new tipo[quantidade  
linhas][quantidade colunas];
```

```
int m[][] = new int[8][4]; // matriz com 8 linhas X  
4 colunas
```

```
public class Exemplo26 {
    public static void main(String[] args) {
        int[][] matriz = new int[3][3];
        Scanner entradaDados = new Scanner(System.in);
        int linhas = matriz.length;
        int colunas = matriz[0].length;
        for (int i = 0; i < linhas; i++) {
            for (int j = 0; j < colunas; j++) {
                System.out.printf("matriz[%d,%d] = ", i, j);
                int x = entradaDados.nextInt();
                matriz[i][j] = x; // Armazena o valor digitado na posição (i,j)
            }
        }
        for (int i = 0; i < linhas; i++) {
            for (int j = 0; j < colunas; j++) {
                System.out.print(matriz[i][j] + " "); // Imprime o elemento seguido de espaço
            }
            System.out.println(); // Quebra de linha após imprimir cada linha da matriz
        }
        entradaDados.close();
    }
}
```


Funções de Matemática

- A classe 'Math' nos proporciona uma série de operações e constantes matemáticas que são facilmente acessadas estaticamente, ou seja, não precisamos instanciar um objeto para podermos usar seus métodos

Potências e raízes

- **pow (base, expoente)** - calcula a potência da base elevada ao expoente.
- **sqrt (número)** - calcula a raiz quadrada de um número
- **cbrt (número)** - calcula a raiz cúbica de um número
- **exp (expoente)** - calcula o valor da constante de Euler (E) elevado ao expoente

```
public static void main(String[] args) {
    Scanner entradaDados = new Scanner(System.in);

    System.out.print("Digite a base: ");
    double base = entradaDados.nextDouble();

    System.out.print("Digite o expoente: ");
    double expoente = entradaDados.nextDouble();

    double potencia = Math.pow(base, expoente);
    System.out.println("Potência (" + base + "^" + expoente + "): " + potencia);

    System.out.print("Digite um número para calcular a raiz quadrada: ");
    double numeroRaiz = entradaDados.nextDouble();
    double raizQuadrada = Math.sqrt(numeroRaiz);
    System.out.println("Raiz quadrada de " + numeroRaiz + ": " + raizQuadrada);

    System.out.print("Digite um número para calcular a raiz cúbica: ");
    double numeroCubo = entradaDados.nextDouble();
    double raizCubica = Math.cbrt(numeroCubo);
    System.out.println("Raiz cúbica de " + numeroCubo + ": " + raizCubica);

    System.out.print("Digite um expoente para calcular E^expoente: ");
    double expValor = entradaDados.nextDouble();
    double resultadoExp = Math.exp(expValor);
    System.out.println("E^" + expValor + ": " + resultadoExp);

    entradaDados.close();
}
```

função	DESCRIÇÃO	EXEMPLO DE USO	RESULTADO
Math.pow(base, expoente)	Calcula a potência da base elevada ao expoente	Math.pow(2, 3)	8.0
Math.sqrt(numero)	Calcula a raiz quadrada	Math.sqrt(16)	4.0
Math.cbrt(numero)	Calcula a raiz cúbica	Math.cbrt(27)	3.0
Math.exp(expoente)	Calcula e elevado ao expoente	Math.exp(1)	2.71828...
Math.abs(numero)	Retorna o valor absoluto	Math.abs(-10)	10
Math.max(a, b)	Retorna o maior entre dois valores	Math.max(5, 7)	7
Math.min(a, b)	Retorna o menor entre dois valores	Math.min(5, 7)	5
Math.round(numero)	Arredonda para o inteiro mais próximo	Math.round(4.6)	5
Math.floor(numero)	Arredonda para baixo (inteiro menor ou igual)	Math.floor(4.9)	4.0
Math.ceil(numero)	Arredonda para cima (inteiro maior ou igual)	Math.ceil(4.1)	5.0
Math.random()	Gera um número aleatório entre 0.0 e 1.0	Math.random()	0.0 a 1.0
Math.toRadians(graus)	Converte graus para radianos	Math.toRadians(180)	3.14159...
Math.toDegrees(radianos)	Converte radianos para graus	Math.toDegrees(Math.PI)	180.0
Math.sin(radianos)	Calcula o seno (em radianos)	Math.sin(Math.PI / 2)	1.0
Math.cos(radianos)	Calcula o cosseno (em radianos)	Math.cos(0)	1.0
Math.tan(radianos)	Calcula a tangente (em radianos)	Math.tan(Math.PI / 4)	1.0

Arredondamento

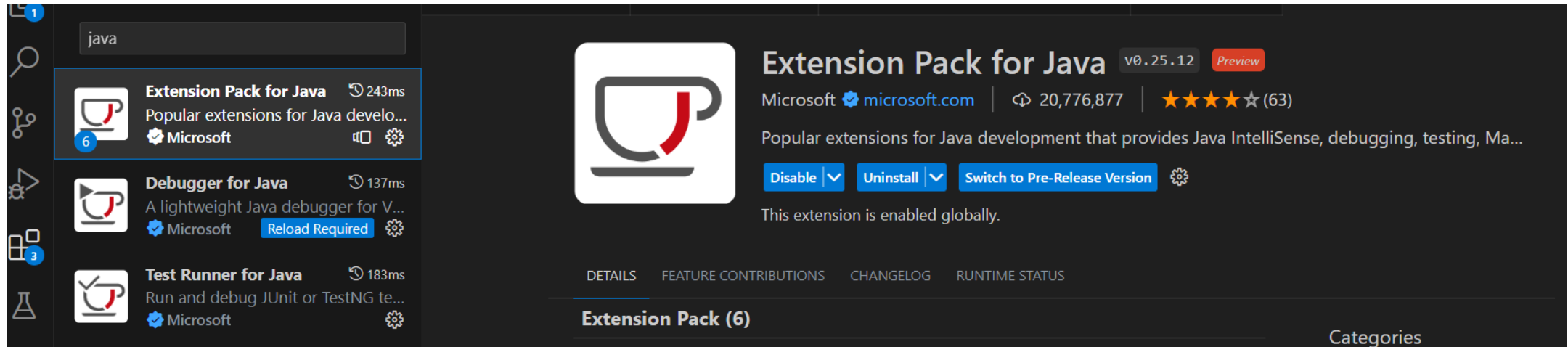
- O arredondamento em Java é usado para ajustar números decimais para valores inteiros ou para um formato mais apropriado.
- A classe Math oferece diferentes métodos para isso, cada um com comportamentos específicos.

Principais métodos de arredondamento

Método	Descrição	Exemplo	Resultado
Math.round(numero)	Arredonda para o inteiro mais próximo. Se a fração for ≥ 0.5 , arredonda para cima; caso contrário, para baixo.	Math.round(4.6)	5
Math.floor(numero)	Arredonda sempre para baixo (menor inteiro mais próximo).	Math.floor(4.9)	4.0
Math.ceil(numero)	Arredonda sempre para cima (maior inteiro mais próximo).	Math.ceil(4.1)	5.0
String.format("%.2f", n)	Formata um número para ter casas decimais específicas (não altera o valor, só a exibição).	String.format("%.2f", 4.5678)	4.57

Vs code

- Instale a extensão pack for java.



Ferramentas online



[Online Java Compiler - online editor \(onlinegdb.com\)](http://onlinegdb.com)



www.jdoodle.com/online-java-compiler/